

Listas Nativas en C#

Manuel Lima - 202201524

 **Tiempo estimado: 20 min**

Universidad San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ingeniería en Ciencias y Sistemas.

Segundo Semestre 2026

Laboratorio – Introducción a la Programación y Computación 2 – Sección P

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad	Al documentar correctamente el uso de cada tipo de lista y elegir la estructura adecuada según el problema, se demuestra responsabilidad profesional en el diseño de software.

1.2 Competencia(s)

Tipo de Competencia	
Competencia General	Aplica estructuras de datos nativas de C# para resolver problemas de almacenamiento y procesamiento de información de forma eficiente.
Competencia Específica	Identifica y utiliza correctamente los tipos de lista nativos en C# (List<T>, Queue<T>, Stack<T>, LinkedList<T>, SortedList<TKey, TValue>) según el contexto del problema a resolver.

2. Palabras Clave

List<T>: Colección genérica dinámica de C# que permite almacenar elementos del mismo tipo y crecer o reducir su tamaño en tiempo de ejecución.

Colección nativa: Estructura de datos incluida en el framework de .NET que no requiere librerías externas para su uso.

3. Introducción

En C#, el manejo eficiente de colecciones de datos es fundamental para el desarrollo de software. A diferencia de los arreglos estáticos, las listas nativas del lenguaje ofrecen flexibilidad, potencia y una amplia variedad de métodos integrados. Esta lectura presenta los tipos de lista más utilizados en C#, su propósito y cuándo conviene emplear cada uno.

4. Contenidos

Tipos de lista en C#

C# ofrece varios tipos de lista nativos dentro del espacio de nombres `System.Collections.Generic`. A continuación se describen los principales:

List<T> es la lista más utilizada en C#. Permite almacenar elementos del mismo tipo, acceder a ellos por índice, agregar, eliminar y buscar elementos de forma dinámica. Es equivalente a un arreglo de tamaño variable.

LinkedList<T> implementa una lista doblemente enlazada. Cada elemento es un nodo que contiene referencias al nodo anterior y al siguiente. Es eficiente para inserciones y eliminaciones frecuentes en cualquier posición de la lista.

Queue<T> implementa el modelo FIFO (First In, First Out): el primer elemento en entrar es el primero en salir. Se usa para gestionar colas de procesamiento, como turnos o tareas pendientes.

Stack<T> implementa el modelo LIFO (Last In, First Out): el último elemento en entrar es el primero en salir. Es útil para historial de acciones, navegación hacia atrás o evaluación de expresiones.

SortedList<TKey, TValue> almacena pares clave-valor manteniendo los elementos ordenados por clave de forma automática. Es útil cuando se necesita recorrer datos en orden sin ordenarlos manualmente.

Comparación de listas nativas

Tipo de Lista	Orden de acceso	Uso recomendado
List<T>	Por índice	Colecciones de uso general con acceso aleatorio
LinkedList<T>	Por nodo enlazado	Inserciones y eliminaciones frecuentes en cualquier posición
Queue<T>	FIFO	Procesamiento de turnos o tareas en cola
Stack<T>	LIFO	Historial de acciones, navegación o evaluación de expresiones
SortedList<TKey, TValue>	Por clave ordenada	Datos que deben mantenerse ordenados por clave

C# cuenta con varios tipos de lista nativos: `List<T>` es la más común, permite duplicados y acceso por índice; `LinkedList<T>` usa nodos enlazados eficientes para inserciones; `Queue<T>` implementa el modelo FIFO; `Stack<T>` implementa el modelo LIFO; y `SortedList<TKey, TValue>` mantiene los elementos ordenados por clave. Cada tipo está diseñado para un caso de uso específico y pertenece al espacio de nombres `System.Collections.Generic`.

Caso de estudio

Se desea almacenar el historial de consultas médicas de un paciente en un sistema hospitalario. Se utilizará `List<string>` para guardar los diagnósticos en orden de ingreso, permitiendo agregar nuevos registros con `Add()`, consultar por índice y recorrer la lista con `foreach`. Si se requiriera procesar las consultas en orden de llegada (primero entrar, primero atender), se usaría `Queue<string>` en su lugar. La elección del tipo de lista depende directamente del comportamiento requerido por la lógica de negocio.

5. Conclusiones

Las listas nativas de C# forman parte del núcleo del framework .NET y ofrecen soluciones eficientes para distintos escenarios de manejo de datos. Comprender sus diferencias permite tomar decisiones de diseño más adecuadas, mejorando el rendimiento y la legibilidad del código. El uso correcto de `List<T>`, `Queue<T>`, `Stack<T>`, `LinkedList<T>` y `SortedList<TKey, TValue>` es una habilidad esencial para cualquier desarrollador en C#.

6. Referencias

1. Lewis, J. (2022). A Quick Tour of Lists in C#. Dev.to. <https://dev.to/jlewis92/a-quick-tour-of-lists-in-c-2id5>
2. NetMentor. (2023). Listas en C#. NetMentor.es. <https://www.netmentor.es/entrada/listas-csharp>